

Числовые поля

Тип Decimal (Money)

Тип числа с фиксированной десятичной точкой. Необходимо использовать для хранения и обработки значений с заданной фиксированной точностью, например при работе с финансовыми данными.



Money vs Decimal

В платформе тип Money — алиас типа Decimal, оставлен для обратной совместимости. Типы ничем не отличаются.

Представление типа

- PostgreSQL — используется тип [numeric](#);
- SQLite — хранится в виде строки;
- Python — тип конвертируется в объект [decimal.Decimal](#);
- C++ — тип реализован на базе библиотеки [libgmp](#).

Сравнение с double

Параметр	Тип double	Тип Decimal	Примечание
Формат хранения	Нормализованное число с двоичной точкой (IEEE 754): $\pm 1.\text{INT}[52 \text{ bits}] * 2^{\text{EXP}[11 \text{ bits}]}$	Целое число с десятичным масштабирующим коэффициентом: $\text{INT} / 10^{\text{SCALE}}$	Двоичная точка (основание экспоненты 2) не позволяет точно описать все значения десятичной системы счисления. Например, значение 0.3 (3/10) в бинарном формате будет представлено как 0.299999999999999988897769... (5404319552844595/18014398509481984). В процессе арифметических операций, например, последовательное суммирование нескольких значений, может привести к накоплению значительной ошибки. Поэтому применять double для финансовых операций не рекомендуется.
Максимальное количество десятичных цифр в числе	Ограничено форматом: 53-битной мантиисой, возможно "тихое" переполнение	Нет ограничений, обусловленных форматом хранения, только реализацией	Ограничение типа double: $\lceil \log_{10}(2^{53}) \rceil = 15$ цифр Ограничения типа Decimal для разных реализаций: • C++ — количество десятичных цифр целой части ограничено памятью компьютера, дробной —

			константой Decimal::MaximalPrecision; • Python — задается контекстом; • PostgreSQL — количество десятичных цифр целой части — до 131072, дробной — до 16383
Количество десятичных чисел в дробной части числа	Не задано — максимально возможное которое позволяет тип. Данное поведение обусловлено форматом хранения.	Фиксированное количество — каждое число имеет заданное количество чисел после запятой, не больше и не меньше.	Неограниченное количество чисел дробной части имеет негативные эффекты на результат арифметических и логических операций. В некоторых случаях такое поведение <u>неуместно</u>. Для типа Decimal имеется режим авто-точности, при котором количество знаков после точки не ограничивается и выбирается автоматически, максимально возможным, в процессе выполнения арифметических операций, чтобы избежать нежелательных округлений.
Точность представления десятичного числа	Приближенная, имеет разные значения на протяжении диапазона значений	Точная, имеет фиксированное значение на всем диапазоне	Данный параметр связан с форматом хранения числа. Бинарная точка и нормализованное/денормализованное представление типа double не позволяют хранить десятичные числа без ошибки. Формат хранения типа Decimal обеспечивает заданную точность представления десятичных чисел на всем диапазоне значений. Тип double "жертвует" точностью в пользу скорости обработки, компактности хранения и охватываемого диапазона значений. Тип Decimal, наоборот, обеспечивает точность в первую очередь, "жертвуя" всем остальным. Данным фактом необходимо руководствоваться при выборе типа.
Наличие специальных значений	Да: -0, NaN, +/-inf	Да: NaN, кроме C++ реализации	Наличие специальных значений требует дополнительных проверок и может привести к неожиданным результатам вычислений. В C++ реализации

			типа Decimal специальные значения отсутствуют, недопустимая операция вызовет исключение.
Операция сравнения	Нельзя сравнивать напрямую	Допустимо прямое сравнение	Из-за приближенной точности double прямое сравнение недопустимо, необходимо вычислить разность и сравнивать с допустимой погрешностью. Наличие отрицательного нуля в double, добавленного для симметрии отрицательного и положительного диапазонов, также может приводить к неожиданным результатам сравнения.
Относительная скорость обработки	Быстро	Медленно	Тип double имеет аппаратную поддержку во всех современных процессорах. Тип Decimal требует выделения памяти в куче и использует программные алгоритмы для реализации арифметических операций поверх стандартного набора процессорных команд для целых чисел. Поэтому не стоит использовать этот тип как замену стандартным типам double или int, если того не требует бизнес-логика.

Сериализация в RPC-вызовах

Исторически сложилось, что по умолчанию тип сериализуется как обычное число двойной точности с плавающей точкой (double). Основные недостатки такого подхода:

- двоичная точка не позволяет точно описать все десятичные значения;
- 53-битная мантисса накладывает ограничение по точности в 15 десятичных цифр, поэтому большие значения могут вызывать переполнение и искажение результата.

Для устранения побочных эффектов double в настройки был добавлен флаг Large decimal, который переключает режим сериализации в строку, что позволяет передавать большие значения без искажений, но на принимающей стороне может потребоваться поддержка "длинной арифметики".

Дополнительные настройки типа в справочниках БЛ

- Поле "Precision" — количество цифр после десятичной точки.
 - Значение 0 — дробная часть отсутствует;
 - Значения > 0 — количество десятичных цифр после фиксированной точки. Если число имеет большую точность, значение округляется в соответствии с правилом [HALF-UP](#);
 - Значения < 0 — авто-точность, количество десятичных цифр после запятой неограниченно.
- Флаг "Large decimal" — флаг сериализации типа, используется для отметки поля, которое потенциально может содержать большие значения.

Python API

Для классов RecordFormat, Record, RecordSet добавлены специальные методы добавления полей Decimal и ArrayDecimal с выставленным флагом формата "Large decimal":

- AddBigMoney, AddArrayBigMoney для RecordFormat, Record;
- AddColBigMoney, AddColArrayBigMoney для RecordSet.

Примеры использования

Данный раздел содержит примеры, относящиеся к платформенной C++ реализации типа Decimal. Для других реализаций необходимо обратиться к оригинальной документации.

Инициализация



Точность по умолчанию

Каждая перегрузка конструктора типа Decimal имеет аргумент точности со значением по умолчанию 2.



Присвоение точности

При инициализации значения существующего Decimal из другого Decimal будет присвоено значение вместе с точностью.

```
1 // Инициализация по умолчанию (0), точность 5 дес. цифр после точки.
2 Decimal d( static_cast< Decimal::DecimalPrecision >( 5 ) );
3
4 // Инициализация из "широкой" строки, точность по умолчанию 2.
5 Decimal d1( L"123.456E-10" );
6
7 // Из "узкой" строки, точность 3.
8 Decimal d2( "-.315159", 3 );
9
10 // Инициализация из StringView, точность 5.
11 Decimal d3( L"-1E+100"_sv, 50 );
12
13 // Инициализация из BinaryStringView, точность 2.
14 Decimal d4( "78946"_sv );
15
16 // Инициализация int, точность 4.
17 Decimal d5( 1, static_cast< Decimal::DecimalPrecision >( 4 ) );
18
19 // Инициализация double, точность 2.
20 Decimal d6( -1.314 );
21
22 // Инициализация float, авто-точность (5).
23 Decimal d7( 3.14159f, Decimal::AutoPrecision );
24
25 // Инициализация целой и дробной частей -2.8162, авто-точность (4).
26 Decimal d8( -2, 8162, Decimal::AutoPrecision );
```

Установка точности

```
1 // = 1.234567, prec: 6.
2 Decimal d(1, 234567, Decimal::AutoPrecision);
3
4 // = 1.235, prec: 3, rounding: half-up.
5 d.SetPrecision( 3 );
6
```

```
7 // = 1.235000, prec: 6, установка точности – необратима.
8 d.SetPrecision( 6 );
```

Арифметические операции

```
1 // a = 1(p3), b = 2(p4), c = 0(p2), d = 0(p2).
2 Decimal a(L"1"_sv, 3), b(2., 4), c, d;
3
4 // c = 3(p4), max prec.
5 c = a + b;
6
7 // c = -1(p4), max prec.
8 c = a - b;
9
10 // c = 2(p4), max prec.
11 c = a * b;
12
13 // c = 0.5(p4), max prec, round: half-up.
14 c = a / b;
15
16 // c = 6(p3), origin prec.
17 c = a + 5;
18
19 // c = -8(p4), origin prec.
20 c = b * -4;
21
22 // c = 10(p3), origin prec.
23 c = 10 / a;
24
25 // c = 10(p3), origin prec.
26 c = a / 0;
27
28 // d = 3(p3), origin prec.
29 d = a; d += b;
30
31 // d = 6(p4), origin prec.
32 d = b; d *= 3;
```

Преобразования

```
1 // = L"1234.00".
2 Decimal( 1234 ).ToString();
3
4 // = L"1.234E3".
5 Decimal( 1234 ).ToExpString();
6
7 // = L"-3.14".
8 Decimal( -3.14159 ).ToString();
9
10 // = L"4.43656457".
11 Decimal( 4.43656457, Decimal::AutoPrecision ).ToString();
12 BinaryStackString< 64 > buf;
13
14 // buf = "-0.2340000000".
15 Decimal( -.234, 10 ).ToString( MakeAllocator( buf ) );
```

```

16
17 // = false.
18 Decimal( 0 ).ToBool();
19
20 // = false.
21 Decimal( -1 ).ToBool();
22
23 // = -123.
24 Decimal( -123 ).ToInteger();
25
26 // = error.
27 Decimal( -123 ).ToUInteger();
28
29 // = -0.234.
30 Decimal( -.234, 10 ).ToDouble();
31
32 // = error.
33 Decimal( "9999999999999999" ).ToInteger();

```

Другие операции

```

1 // = 1 / 10 = 0.10.
2 Decimal( 10 ).Reverse();
3
4 // = 1.
5 Decimal( -1 ).Abs();
6
7 // = 2.
8 Decimal( 1.23 ).Ceil();
9
10 // = -1.
11 Decimal( -1.23 ).Ceil();
12
13 // = 2.
14 Decimal( 2.99 ).Floor();
15
16 // = -3.
17 Decimal( -2.99 ).Floor();
18 Decimal d( -2, 4 );
19
20 // -2.00 + -0.40 == -2.40.
21 d.IntegerPart() + d.FractionPart() == d
22
23 // int_fract = 60, формула для подстановки floor(1.6) + int_fract /
24 // scale = 1 + 60 / 100. = 1.60.
25 Decimal( 1.6 ).IntFract();
26
27 // int_fract = 40, формула для подстановки floor(-1.6) + int_fract /
28 // scale = -2 + 40 / 100 = -1.6.
29 Decimal( -1.6 ).IntFract();
30
31 // fract = 0.6, формула для подстановки floor(1.6) + fract = 1 + 0.6 =
32 // 1.6.
33 Decimal( 1.6 ).Fract();
34
35

```

```
32 // fract = 0.4, формула для подстановки floor(-1.6) + fract = -2 + 0.4  
   = -1.6.  
33 Decimal( -1.6 ).Fract()
```